

MLOps Pipeline: Cats vs Dogs Image Classification

GitHub Repository: https://github.com/himajajaddu/MLOPS_assignment_2

Project Demo Video:

<https://drive.google.com/file/d/1SngcW8YZ5EKk3udamfqvMfc7fOmyoCal/view>

Project Objective

This project builds a reproducible end-to-end MLOps pipeline for binary image classification (Cats vs Dogs) tailored for a pet adoption platform. It demonstrates training, experiment tracking, data versioning, containerization, CI/CD automation, deployment, and monitoring.

Dataset & Preprocessing

Source: Kaggle Cats vs Dogs Dataset

Preprocessing Steps:

- Resize images and normalize to RGB format
- Split into Train / Validation / Test sets
- Apply augmentation for improved generalization

MLOps Architecture

M1: Model Development & Tracking → PyTorch, MLflow

M2: Packaging & Containerization → FastAPI, Docker

M3: Continuous Integration → GitHub Actions, Pytest

M4: Continuous Deployment → Docker Compose

M5: Monitoring & Logging → Prometheus metrics & logging

Project Structure

app/ → FastAPI inference service

data/ → Dataset (DVC tracked)

deployment/ → Docker Compose setup

models/ → Saved trained model

reports/ → Evaluation outputs

src/ → Training & preprocessing code

.github/workflows/ → CI pipeline

Dockerfile → Container configuration

Model Training & Experiment Tracking

Training Command:

```
python -m src.train --data_dir data/raw --epochs 3
```

Outputs:

- Trained model → models/cats_dogs_model.pt
- Metrics & logs tracked in MLflow

FastAPI Service

Start server: `uvicorn app.main:app --reload`

GET /health → service health

POST /predict → image classification

GET /metrics → Prometheus metrics

Docker & Deployment

Build image: `docker build -t cats-dogs-classifier .`

Run container: `docker run -p 8000:8000 cats-dogs-classifier`

Deploy: `docker-compose up -d` (from deployment folder)

CI/CD Pipeline

CI (GitHub Actions):

- Install dependencies
- Run tests
- Build Docker image
- Validate build

CD Flow:

- Pull latest image
- Restart service
- Health check validation

Monitoring & Metrics

- Logs include request info, latency, and errors

- Prometheus metrics exposed at /metrics
- Tracks request count, response time, and error rate

Execution Evidence

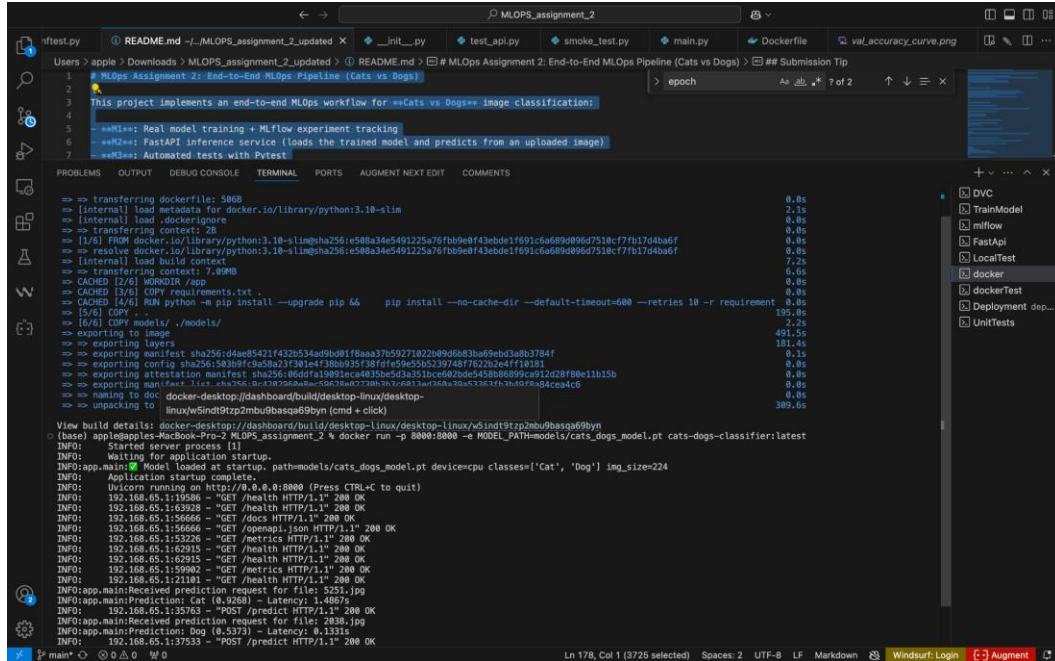


Figure: Docker build and container startup

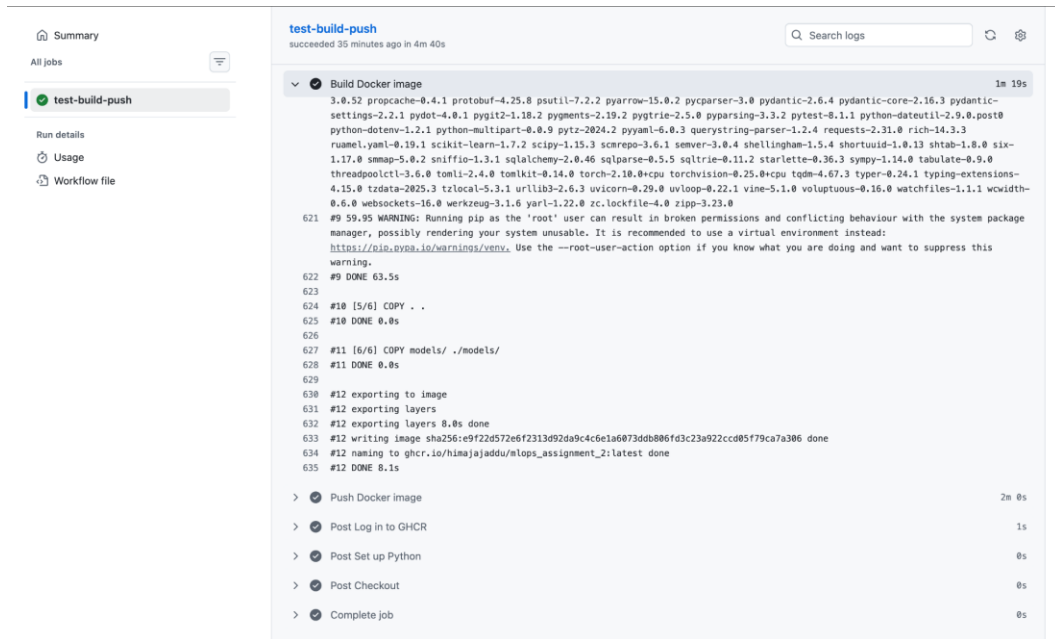


Figure: GitHub Actions CI pipeline success

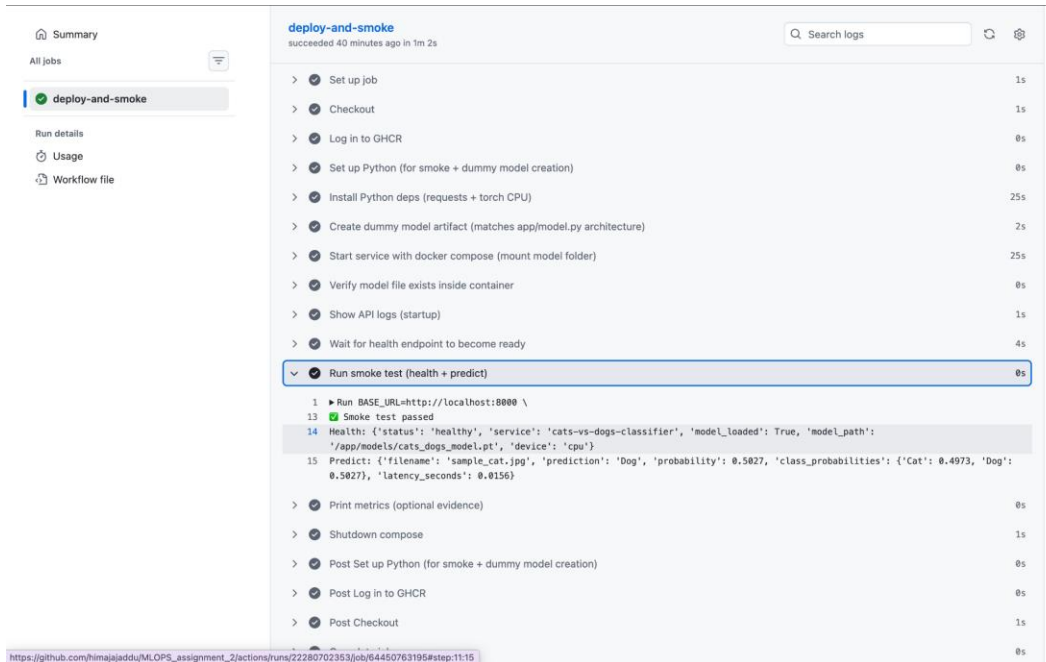


Figure: CD deployment & health check

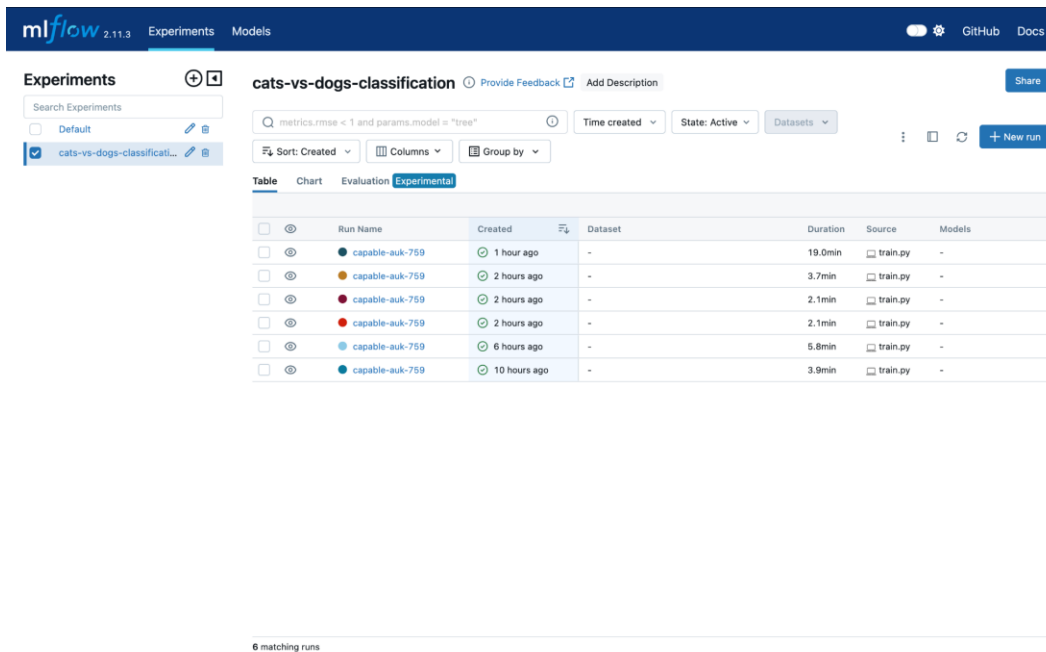


Figure: MLflow experiment tracking

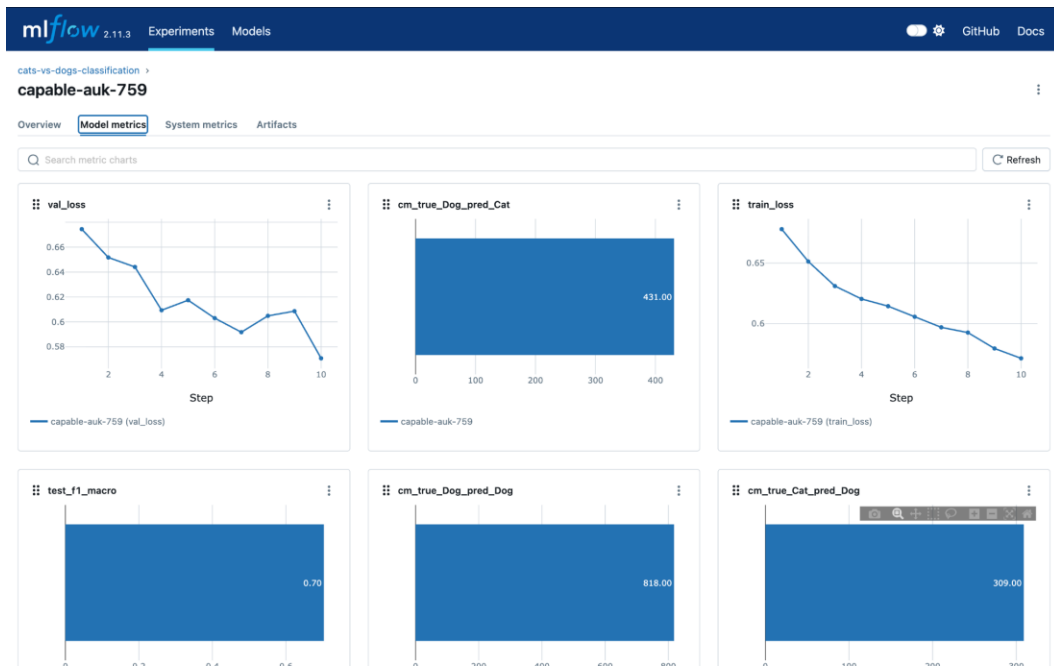


Figure: Model evaluation metrics

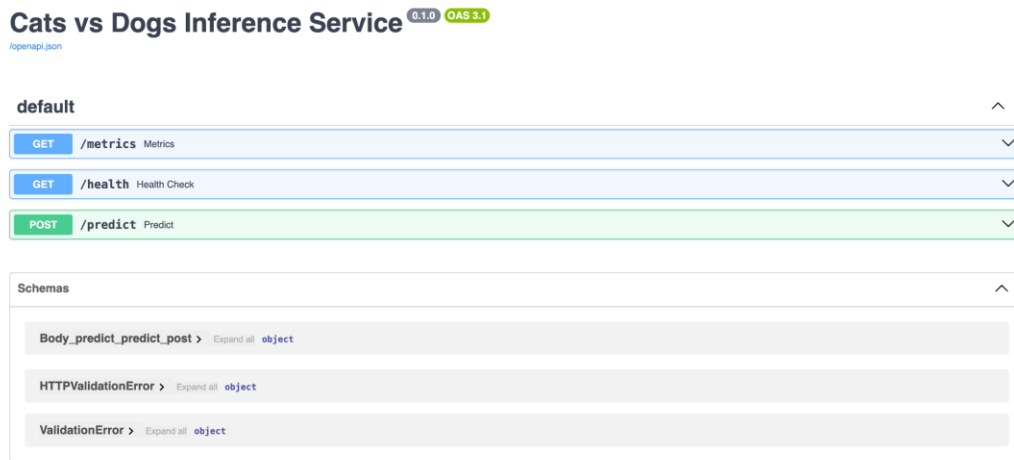


Figure: API health & prediction curl tests

Conclusion

This project demonstrates a production-ready ML system integrating reproducibility, automation, deployment, and monitoring. The pipeline ensures scalable and reliable model serving suitable for real-world adoption platforms.